



汇编语言

2 软件开发与执行过程

张青

ieqzhang@zzu.edu.cn



教学内容

- 复习系统概述

教材章节：第1.2-1.3

慕课：1.3

- ▶ 硬件组成

- ▶ 存储器组织

- ▶ 数据单位

- 软件开发过程

- 汇编语言程序格式

教材章节：第1.4

慕课：2

- ▶ 开发步骤及工具

- ▶ 程序执行过程

- ▶ 汇编语句格式和程序格式

- 数据表示导学

教材章节：第2.1-2.3

慕课：3.1-3.4,3.6

- ▶ 数制、编码

- ▶ GAS伪指令

学习目标



- 能阐述程序开发步骤及使用的软件
- 能叙述编译器、汇编器、连接器的作用
- 能利用GCC实现C语言转换为汇编语言、目标文件和可执行文件
- 能阐述可执行程序的执行过程
- 能阐述汇编语言格式组成
- 能写出GAS汇编语言程序框架
- 养成细致严谨的态度，注意C语言与汇编语言之间的联系

课后作业



教学要点

- 厚山学堂/学习通学习:

- 视频: 1.3
- 线上讨论: 1.4
- 单元测试: 1.5

截止时间: 3月15日23: 55, 可提交3次

- ✓ 认识汇编语言
- ✓ 计算机系统概述
- ✓ 指令集架构

“吐槽”汇编语言程序设计慕课

学情统计-3班4班



章节测验 (共发布4个)

平均完成进度

0 个

■ 学生最快进度 2个

■ 学生最慢进度 0个

未获得分数：学生未提交章节测验，或章节测验未被批阅

讨论

话题数

2 个

回复数

156 个

平均分

98 分



■ 60 分以下 0 人

■ 60~79 分 0 人

■ 80~100 分 35 人

■ 未获得分数 67 人

近七日话题活跃度

43%

0% 比上周

学情统计-9班10班



章节测验 (共发布4个)

平均完成进度

0 个

- 学生最快进度 3个
- 学生最慢进度 0个

未获得分数：学生未提交章节测验，或章节测验未被批阅

讨论

话题数

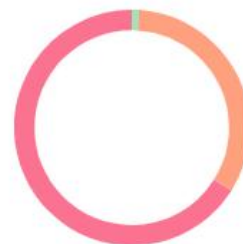
2 个

回复数

182 个

平均分

98 分



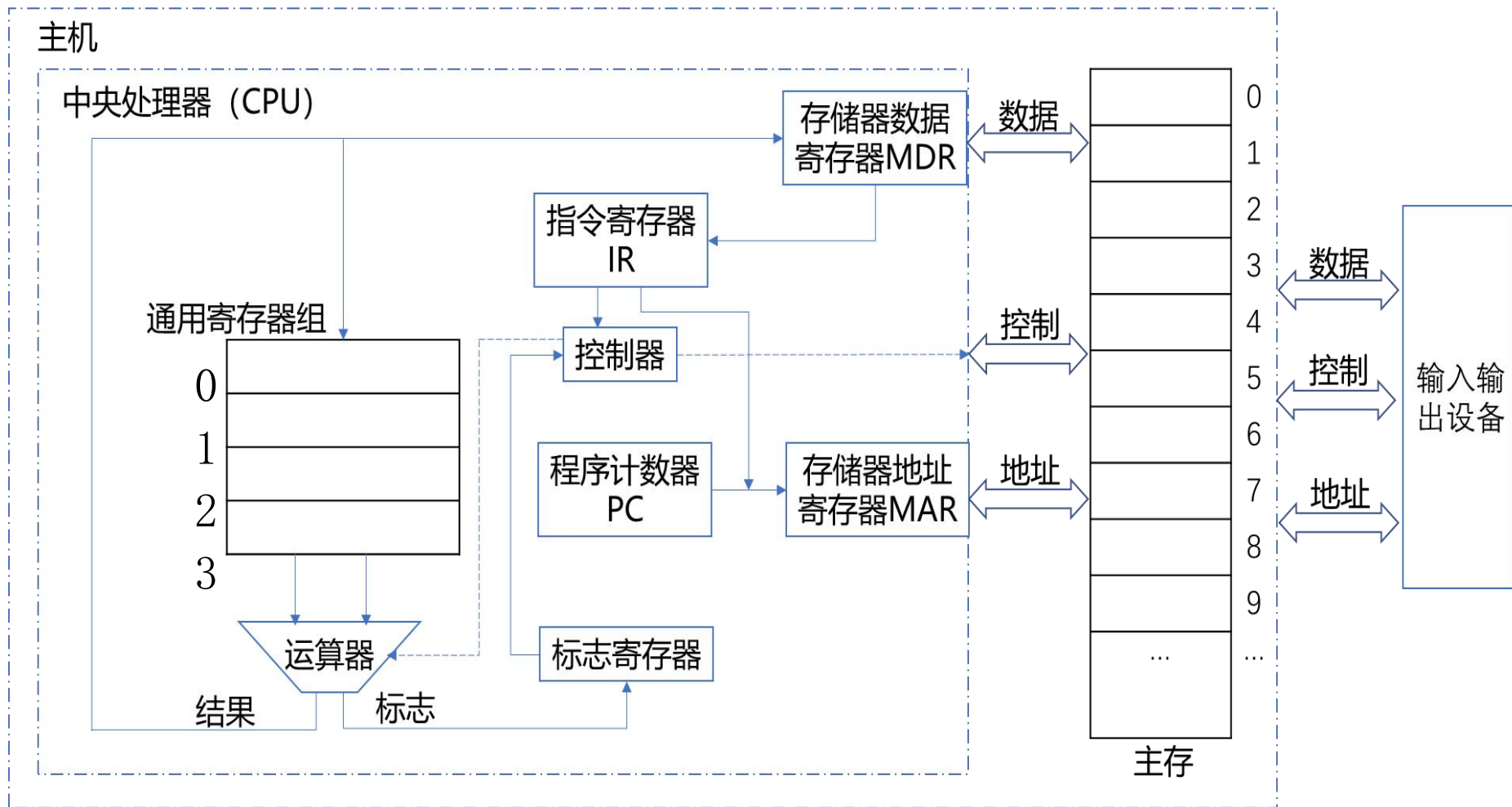
- 60 分以下 0 人
- 60~79 分 1 人
- 80~100 分 34 人
- 未获得分数 68 人

近七日话题活跃度

46%

0% 比上周

计算机硬件结构



存储器地址空间



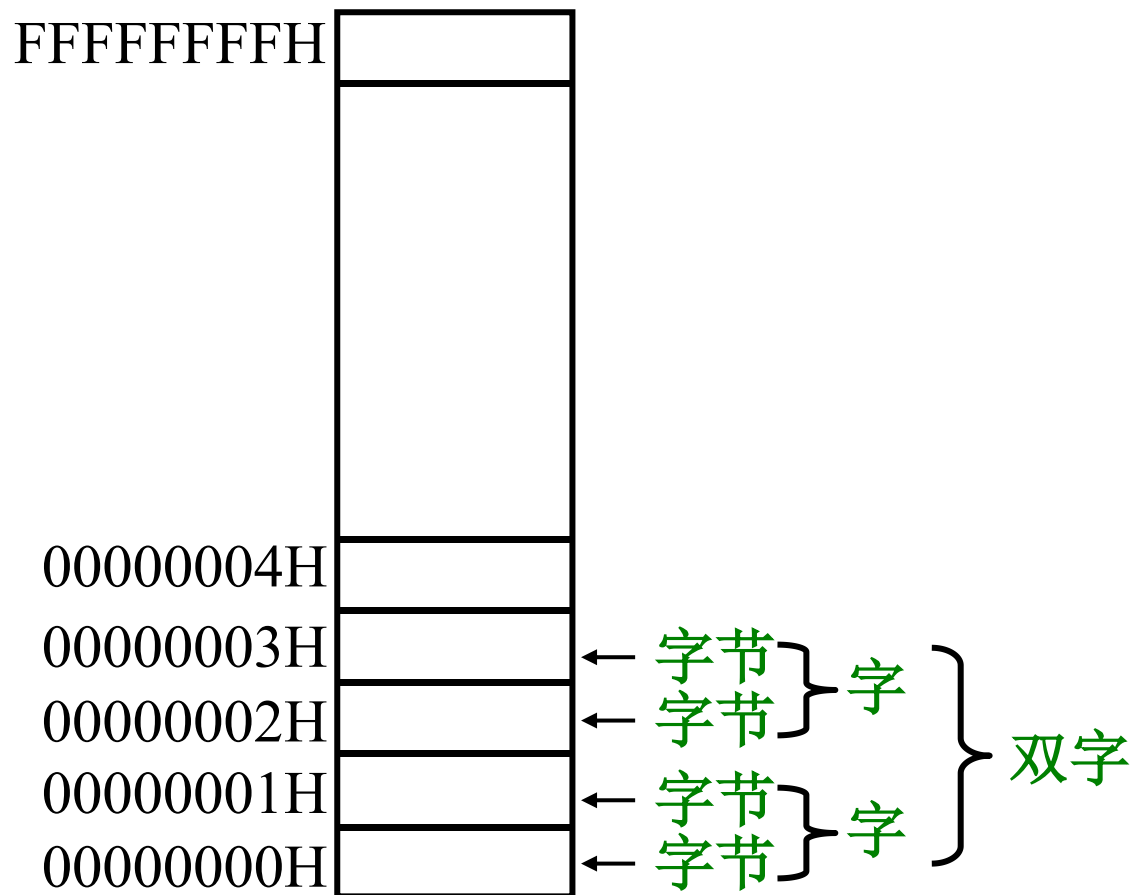
- 物理地址空间从0开始顺序编排，直到处理器支持的最大存储单元

- 8086处理器支持存储器：

00000H ~ FFFFFH

- IA-32处理器支持存储器：

00000000H ~ FFFFFFFFH



计算机系统的层次结构



指令集体系架构ISA



- 指令集体系架构（Instruction Set Architecture, ISA）也叫做计算机体系结构。
- IAS位于硬件和软件的交界面上，是整个计算机系统的核心部分，是构建计算机硬件和软件的规范。
 - 硬件所具有的功能特性都由ISA集中体现
 - 软件要通过ISA在硬件上执行
- 目前典型的指令集架构：
 - CISC（Complex Instruction Set Computer, 复杂指令集计算机）
 - RISC（Reduced Instruction Set Computer, 精简指令集计算机）



邀请码: 12570104

APP首页右上角输入



该邀请码2026年08月26日前有效

计算机类2024级3班4班

复习系统概述

随堂练习(学习通): 系统概述随堂练习

邀请码: 97974770

APP首页右上角输入



复习系统概述

随堂练习(学习通): 系统概述随堂练习



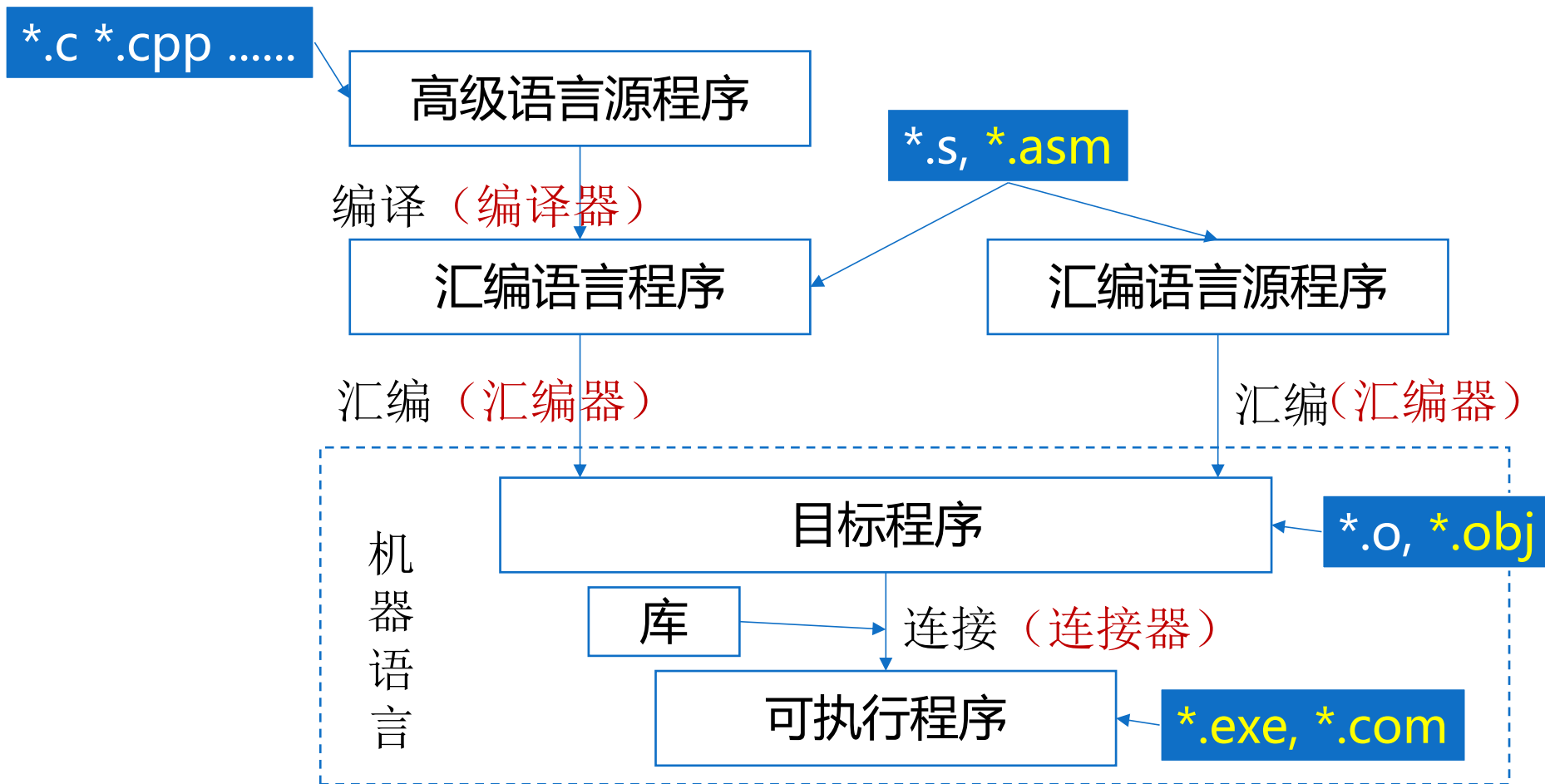
该邀请码2026年08月26日前有效

计算机类2024级9班10班



软件开发过程

程序开发过程



Windows系统下的默认扩展名

〔例1-2〕 编写一个C语言程序，用于显示两数之和



```
//eg0102.c
#include <stdio.h>
int add(int x, int y){
    int t;
    t = x+y;
    return t;
}
int main(){
    printf("%d", add(100,256));
    return 0;
}
```

可执行文件的开发过程

- 使用下列命令可以生成可执行文件：

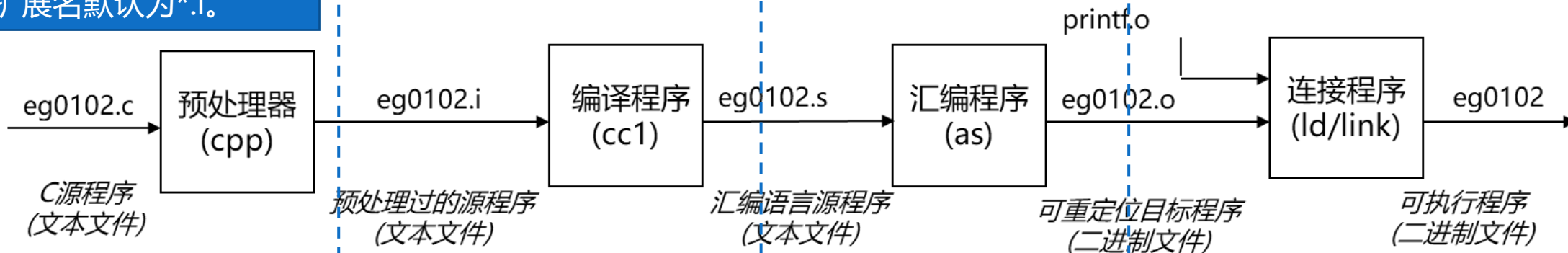
```
gcc -Og -o eg0102 eg0102.c
```

```
gcc -E -o eg0102.i eg0102.c
```

预处理阶段对源程序中的预处理语言进行处理，输出还是源程序文件，扩展名默认为*.i。

```
gcc -c -o eg0102s.o eg0102.s
```

汇编阶段对汇编语言源程序进行汇编生成定位目标程序（二进制文件），扩展名默认为*.o



编译阶段对预处理后的源程序进行编译，生成汇编语言源程序文件，扩展名默认为*.s。

连接阶段将多个可重定位目标程序和标准函数库合并为可执行文件

```
gcc -S -Og -masm=intel -o eg0102.s eg0102.c
```

```
gcc -o eg0102 eg0102.o
```


使用快速开发命令



```
G:\MLL\汇编语言\数字化教材\章节\初稿v3\ZZUGAS>C2S eg0102
```

编译

```
G:\MLL\汇编语言\数字化教材\章节\初稿v3\ZZUGAS>echo off
```

```
G:\MLL\汇编语言\数字化教材\章节\初稿v3\ZZUGAS>S2E eg0102
```

汇编连接

```
G:\MLL\汇编语言\数字化教材\章节\初稿v3\ZZUGAS>echo off
```

```
找不到 G:\MLL\汇编语言\数字化教材\章节\初稿v3\ZZUGAS\eg0102.lst
```

```
找不到 G:\MLL\汇编语言\数字化教材\章节\初稿v3\ZZUGAS\eg0102.o
```

```
驱动器 G 中的卷没有标签。
```

```
卷的序列号是 6C05-FBDB
```

```
G:\MLL\汇编语言\数字化教材\章节\初稿v3\ZZUGAS 的目录
```

```
2026/02/11 16:11          61,271 eg0102.exe
                1 个文件          61,271 字节
                0 个目录 4,447,854,592 可用字节
```

```
G:\MLL\汇编语言\数字化教材\章节\初稿v3\ZZUGAS>eg0102
```

执行

```
356
```

配套软件：厚山学堂-汇编语言1.3.2.3课程配套软件





汇编语言程序格式

- gcc **-Og -S -masm=intel** -o eg0102.s eg0102.c

```
.file "eg0102.c"
```

```
.intel_syntax noprefix
```

```
.text
```

```
.globl add
```

```
.def add; .scl 2; .type 32; .endef
```

```
.seh_proc add
```

标号: 真指令助记符

操作数

注释

add:

push

rbp

#x+y

mov

rbp, rsp

sub

rsp, 16

mov

DWORD PTR 16[rbp], ecx

/* 省略 */

add

eax, edx

mov

DWORD PTR -4[rbp], eax

mov

eax, DWORD PTR -4[rbp]

add

rsp, 16

pop

rbp

ret

.section .rdata,"dr"

#定义只读数据段

标号: 伪指令助记符

参数

注释

.LC0: **.ascii**

"%d\0"

/*定义字符串*/

main:

/*省略*/

lea

mov

call

rax, **.LC0**[rip]

rcx, rax

printf

课堂活动

改成 "x+y=%d\0"
结果如何?

标号: 助记符 操作数 #注释

表示处理器指令在主存中的逻辑地址的标识符

表示功能的记忆符号
真指令：处理器命令
伪指令：汇编器命令

参数，初值表

- 自定义标识符：
 - 以字母或下划线开始，不能以数字开头，大小写敏感
 - 由字母、数字及规定的特殊符号（“_”、“.”、“\$”）组成
 - 自定义标识符必须是唯一的，不能用汇编程序的保留字
 - 应尽量具有描述性并易于理解，不建议使用特殊符号开头

程序框架



```
/* AS汇编语言基本框架 */
```

```
.intel_syntax noprefix # 采用Intel语法, 面向X86处理器时使用
```

```
.data # 定义数据段
```

```
# 数据定义 (数据待填)
```

```
.text # 定义代码段
```

```
.globl main
```

```
main: # 程序执行起始位置
```

```
#主程序 (指令待填)
```

```
ret # 程序正常执行结束
```

```
#子程序 (指令待填)
```


例1-3：信息显示程序（汇编语言）



```
/* eg0103.s in Windows Terminal for GCC_64*/  
.intel_syntax noprefix  
.section .data  
    msg: .asciz "Hello, World!\n"  
.section .text  
.globl main  
main:  
    lea rax, msg[rip]    # 入口参数: 字符串首地址  
    call dispmsg        # 调用自定义子程序  
    xor eax,eax         # 程序结束返回  
    ret
```

输入输出子程序库

- 汇编程序通常不提供任何函数或程序库
- 必须利用操作系统的编程资源
- 本书配套键盘输入和显示器输出的I/O子程序

子程序名	DISPMSG
入口参数	RAX=字符串地址
功能说明	显示字符串（以0结尾）

信息显示程序汇编连接



```
gcc -o eg0103 ./progs/eg0103.s ./lib/winio64.a
```

汇编:

```
as -o eg0103.o ./progs/eg0103.s
```

连接:

```
gcc -o eg0103 eg0103.o ./lib/winio64.a
```

```
gcc -o eg0103 ./progs/eg0103.s ./lib/io_linux64.a
```

Windows+x86-64



Linux+x86-64



常用输入输出子程序



子程序名	功能说明
DISPMMSG	显示字符串（以0结尾）
DISPC	显示一个字符
DISPCRLF	光标回车换行，到下一行首个位置
DISPRD	显示8个32位通用寄存器内容
DISPRF	显示6个状态标志的状态
DISPHD(X)	以十六进制形式显示8(16)位数据
DISPUID(X)	显示32(64)位无符号十进制整数
DISPSID(X)	显示32(64)位有符号十进制整数
READMSG	输入一个字符串（回车结束）
READC	输入一个字符（回显）



数据表示导学

教材章节：第2.1-2.3

慕课：3.1-3.4, 3.6

- ▶ 数制、编码
- ▶ 数据定义伪指令

数制



- 十进制
- 十六进制
- 二进制
- 八进制
- 相互转换

$$\begin{aligned} & 10, 16, 2, 8 \\ N_d &= k_{n-1} k_{n-2} \dots k_0 . k_{-1} k_{-2} \dots k_m \\ &= \sum_{i=-m}^{n-1} k_i d^i \end{aligned}$$

- 整数编码
 - 原码, 反码, 补码, BCD码
- 字符
 - ASCII, Unicode
- (可选) 浮点数
 - IEEE754

机器数多重解释：0x81323334的多视角解码



编码类型	解释方法	真值
无符号数	直接转换	2,167,550,772
原码	最高位符号，其余为绝对值	-20,040,244
反码	符号位不变，其余取反	-2,127,443,403
补码	符号位不变，取反+1	-2,127,443,404
BCD码	每4位转十进制	81,323,334
IEEE 754	s/e/M分段计算	-3.274×10^{-38}
ASCII	每字节转字符	"?234" (不可显示)
核心启示： • 相同二进制，不同解释 → 不同含义 • 数据类型决定解码方式		

伪指令	大小	C类型	示例
.byte	1字节	char	.byte 65 → 'A'
.word	2字节	short	.word -1 → 0xFFFF
.long	4字节	int	.long -1 → 0xFFFFFFFF
.quad	8字节	long/指针	.quad str → 地址
.ascii	N字节	char[]	.ascii "AB\0"

教学要点

- 慕课学习：
 - 视频：3.1-3.4
 - 单元测试：2.7 (必做)
 - 讨论：2.8
 - 第一章作业：2.9

- ✓ 开发步骤及工具
- ✓ 汇编语句格式和程序格式